

```
In [1]: import pandas as pd #always keep all imports at top
        #allows us to show a correlation-heatmap
        import seaborn as sns #seaborn allows us to make the heatmap and correlations like
        import matplotlib.pyplot as plt #matplotlib allows us to change form of the chart,
```

```
In [2]: df = pd.read_csv(r"C:\Users\Rashid\Downloads\sales_data (1).csv") #make sure you
```

```
In [3]: df #for tables, dont use print bcus with print its more complicated
```

Out[3]:

	Transaction_ID	Date	Customer_ID	Product	Category	Quantity	Price	Total_Amount	F
0	1001.0	2024-01-05	C001	Laptop	Electronics	1.0	800.0	NaN	
1	1002.0	2024-01-10	C002	Smartphone	Electronics	2.0	600.0	1200.0	
2	1003.0	2024-01-12	C003	Headphones	Electronics	1.0	100.0	100.0	
3	1004.0	2024-02-05	C004	Tablet	Electronics	1.0	500.0	500.0	
4	1005.0	2024-02-08	C005	Book	Books	3.0	20.0	60.0	
5	1006.0	2024-02-10	C001	Laptop	Electronics	1.0	800.0	800.0	
6	1007.0	2024-03-15	C006	Shoes	Clothing	2.0	50.0	100.0	
7	1008.0	2024-03-18	C007	T-Shirt	Clothing	1.0	25.0	25.0	
8	1009.0	2024-03-20	C008	Smartwatch	Electronics	1.0	200.0	200.0	
9	1010.0	2024-04-01	C009	Book	Books	2.0	20.0	40.0	
10	1011.0	2024-04-05	C002	Smartphone	Electronics	2.0	600.0	1200.0	
11	1012.0	2024-04-10	C010	Tablet	Electronics	1.0	500.0	500.0	
12	1013.0	2024-05-01	C011	Shoes	Clothing	1.0	50.0	50.0	
13	1014.0	2024-05-05	C012	Headphones	Electronics	1.0	100.0	100.0	
14	1015.0	2024-05-08	C013	Laptop	Electronics	1.0	800.0	800.0	
15	1016.0	2024-05-10	C014	T-Shirt	Clothing	3.0	25.0	75.0	
16	1017.0	2024-06-01	C015	Smartwatch	Electronics	1.0	200.0	200.0	
17	1018.0	2024-06-05	C016	Book	Books	4.0	20.0	80.0	
18	1019.0	2024-06-08	C017	Smartphone	Electronics	1.0	600.0	600.0	
19	1020.0	2024-06-10	C018	Tablet	Electronics	1.0	500.0	500.0	
20	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	

21 rows × 23 columns



```
In [4]: df = df.dropna(axis=1, how='all') #if nan is in the whole column it will get rid of df
```

Out[4]:

	Transaction_ID	Date	Customer_ID	Product	Category	Quantity	Price	Total_Amount	F
0	1001.0	2024-01-05	C001	Laptop	Electronics	1.0	800.0	NaN	
1	1002.0	2024-01-10	C002	Smartphone	Electronics	2.0	600.0	1200.0	
2	1003.0	2024-01-12	C003	Headphones	Electronics	1.0	100.0	100.0	
3	1004.0	2024-02-05	C004	Tablet	Electronics	1.0	500.0	500.0	
4	1005.0	2024-02-08	C005	Book	Books	3.0	20.0	60.0	
5	1006.0	2024-02-10	C001	Laptop	Electronics	1.0	800.0	800.0	
6	1007.0	2024-03-15	C006	Shoes	Clothing	2.0	50.0	100.0	
7	1008.0	2024-03-18	C007	T-Shirt	Clothing	1.0	25.0	25.0	
8	1009.0	2024-03-20	C008	Smartwatch	Electronics	1.0	200.0	200.0	
9	1010.0	2024-04-01	C009	Book	Books	2.0	20.0	40.0	
10	1011.0	2024-04-05	C002	Smartphone	Electronics	2.0	600.0	1200.0	
11	1012.0	2024-04-10	C010	Tablet	Electronics	1.0	500.0	500.0	
12	1013.0	2024-05-01	C011	Shoes	Clothing	1.0	50.0	50.0	
13	1014.0	2024-05-05	C012	Headphones	Electronics	1.0	100.0	100.0	
14	1015.0	2024-05-08	C013	Laptop	Electronics	1.0	800.0	800.0	
15	1016.0	2024-05-10	C014	T-Shirt	Clothing	3.0	25.0	75.0	
16	1017.0	2024-06-01	C015	Smartwatch	Electronics	1.0	200.0	200.0	
17	1018.0	2024-06-05	C016	Book	Books	4.0	20.0	80.0	
18	1019.0	2024-06-08	C017	Smartphone	Electronics	1.0	600.0	600.0	
19	1020.0	2024-06-10	C018	Tablet	Electronics	1.0	500.0	500.0	
20	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	

```
In [5]: df = df.dropna(how='all') #gets rid of any nan rows
df
```

Out[5]:

	Transaction_ID	Date	Customer_ID	Product	Category	Quantity	Price	Total_Amount	F
0	1001.0	2024-01-05	C001	Laptop	Electronics	1.0	800.0	NaN	
1	1002.0	2024-01-10	C002	Smartphone	Electronics	2.0	600.0	1200.0	
2	1003.0	2024-01-12	C003	Headphones	Electronics	1.0	100.0	100.0	
3	1004.0	2024-02-05	C004	Tablet	Electronics	1.0	500.0	500.0	
4	1005.0	2024-02-08	C005	Book	Books	3.0	20.0	60.0	
5	1006.0	2024-02-10	C001	Laptop	Electronics	1.0	800.0	800.0	
6	1007.0	2024-03-15	C006	Shoes	Clothing	2.0	50.0	100.0	
7	1008.0	2024-03-18	C007	T-Shirt	Clothing	1.0	25.0	25.0	
8	1009.0	2024-03-20	C008	Smartwatch	Electronics	1.0	200.0	200.0	
9	1010.0	2024-04-01	C009	Book	Books	2.0	20.0	40.0	
10	1011.0	2024-04-05	C002	Smartphone	Electronics	2.0	600.0	1200.0	
11	1012.0	2024-04-10	C010	Tablet	Electronics	1.0	500.0	500.0	
12	1013.0	2024-05-01	C011	Shoes	Clothing	1.0	50.0	50.0	
13	1014.0	2024-05-05	C012	Headphones	Electronics	1.0	100.0	100.0	
14	1015.0	2024-05-08	C013	Laptop	Electronics	1.0	800.0	800.0	
15	1016.0	2024-05-10	C014	T-Shirt	Clothing	3.0	25.0	75.0	
16	1017.0	2024-06-01	C015	Smartwatch	Electronics	1.0	200.0	200.0	
17	1018.0	2024-06-05	C016	Book	Books	4.0	20.0	80.0	
18	1019.0	2024-06-08	C017	Smartphone	Electronics	1.0	600.0	600.0	
19	1020.0	2024-06-10	C018	Tablet	Electronics	1.0	500.0	500.0	

```
In [6]: #inspecting data:
df.head()
```

```
#This shows you the first 5 rows of your data (like the top 5 rows in Excel).so you
(df.info())
# gives summary of whole table ( DataFrame)(df)), tells us no. of rows, columns, co
#useful: Helps spot data problems early (e.g., wrong data type, missing values). c
(df.describe())
#gives quick stats about numerical columns in dataset.inc:
#count - how many values there are
#mean - average
#std - standard deviation (spread of the values)
#min - minimum value
#25%, 50%, 75% - quartiles (like median, and ranges)
#max - highest value
#useful:understand how numbers are distributed, Detect outliers/errors (like product
```

```
<class 'pandas.core.frame.DataFrame'>
Int64Index: 20 entries, 0 to 19
Data columns (total 11 columns):
#   Column                Non-Null Count  Dtype
---  -
0   Transaction_ID         20 non-null    float64
1   Date                   20 non-null    object
2   Customer_ID            20 non-null    object
3   Product                 20 non-null    object
4   Category                20 non-null    object
5   Quantity                20 non-null    float64
6   Price                   20 non-null    float64
7   Total_Amount            19 non-null    float64
8   Payment_Method          20 non-null    object
9   Region                  20 non-null    object
10  Advertising_Spend       20 non-null    float64
dtypes: float64(5), object(6)
memory usage: 1.9+ KB
```

```
Out[6]:
```

	Transaction_ID	Quantity	Price	Total_Amount	Advertising_Spend
count	20.000000	20.000000	20.000000	19.000000	20.000000
mean	1010.50000	1.550000	325.500000	375.263158	132.750000
std	5.91608	0.887041	302.484884	389.067524	106.233048
min	1001.00000	1.000000	20.000000	25.000000	10.000000
25%	1005.75000	1.000000	43.750000	77.500000	23.750000
50%	1010.50000	1.000000	200.000000	200.000000	120.000000
75%	1015.25000	2.000000	600.000000	550.000000	212.500000
max	1020.00000	4.000000	800.000000	1200.000000	300.000000

```
In [7]: #Remove duplicates:
df = df.drop_duplicates() #says create a new variable (df) so same table but with
df

#Handle missing values:
print("Missing Values: ", df.isnull().sum()) #red=string, print needed bcus of stri

df= df.fillna(df['Total_Amount'].mean()) #if there are missing values replace it th
df
```

```
Missing Values: Transaction_ID    0
Date                0
Customer_ID        0
Product            0
Category           0
Quantity           0
Price              0
Total_Amount       1
Payment_Method     0
Region             0
Advertising_Spend  0
dtype: int64
```

Out[7]:

	Transaction_ID	Date	Customer_ID	Product	Category	Quantity	Price	Total_Amount	F
0	1001.0	2024-01-05	C001	Laptop	Electronics	1.0	800.0	375.263158	
1	1002.0	2024-01-10	C002	Smartphone	Electronics	2.0	600.0	1200.000000	
2	1003.0	2024-01-12	C003	Headphones	Electronics	1.0	100.0	100.000000	
3	1004.0	2024-02-05	C004	Tablet	Electronics	1.0	500.0	500.000000	
4	1005.0	2024-02-08	C005	Book	Books	3.0	20.0	60.000000	
5	1006.0	2024-02-10	C001	Laptop	Electronics	1.0	800.0	800.000000	
6	1007.0	2024-03-15	C006	Shoes	Clothing	2.0	50.0	100.000000	
7	1008.0	2024-03-18	C007	T-Shirt	Clothing	1.0	25.0	25.000000	
8	1009.0	2024-03-20	C008	Smartwatch	Electronics	1.0	200.0	200.000000	
9	1010.0	2024-04-01	C009	Book	Books	2.0	20.0	40.000000	
10	1011.0	2024-04-05	C002	Smartphone	Electronics	2.0	600.0	1200.000000	
11	1012.0	2024-04-10	C010	Tablet	Electronics	1.0	500.0	500.000000	
12	1013.0	2024-05-01	C011	Shoes	Clothing	1.0	50.0	50.000000	
13	1014.0	2024-05-05	C012	Headphones	Electronics	1.0	100.0	100.000000	
14	1015.0	2024-05-08	C013	Laptop	Electronics	1.0	800.0	800.000000	
15	1016.0	2024-05-10	C014	T-Shirt	Clothing	3.0	25.0	75.000000	
16	1017.0	2024-06-01	C015	Smartwatch	Electronics	1.0	200.0	200.000000	
17	1018.0	2024-06-05	C016	Book	Books	4.0	20.0	80.000000	
18	1019.0	2024-06-08	C017	Smartphone	Electronics	1.0	600.0	600.000000	
19	1020.0	2024-06-10	C018	Tablet	Electronics	1.0	500.0	500.000000	

In [8]:

```
#Impute categorical data by replacing missing values with the most frequent category
# Step 1: Clean column names (remove extra spaces)
df.columns = df.columns.str.strip() #remove unnecessary things ex extra commas, spaces

# Step 2: Select all categorical (text) columns
categorical_cols = df.select_dtypes(include='object').columns #sees which of the columns are categorical
```

```
# Step 3: For each column, fill missing values with the most frequent value (mode)
for col in categorical_cols:
    if df[col].isnull().sum() > 0: # counts how many are missing values
        most_common = df[col].mode()[0] # gets most common value (mode) of each of
        df[col].fillna(most_common, inplace=True) #wherever there are missing values
df
```


Out[8]:

	Transaction_ID	Date	Customer_ID	Product	Category	Quantity	Price	Total_Amount	F
0	1001.0	2024-01-05	C001	Laptop	Electronics	1.0	800.0	375.263158	
1	1002.0	2024-01-10	C002	Smartphone	Electronics	2.0	600.0	1200.000000	
2	1003.0	2024-01-12	C003	Headphones	Electronics	1.0	100.0	100.000000	
3	1004.0	2024-02-05	C004	Tablet	Electronics	1.0	500.0	500.000000	
4	1005.0	2024-02-08	C005	Book	Books	3.0	20.0	60.000000	
5	1006.0	2024-02-10	C001	Laptop	Electronics	1.0	800.0	800.000000	
6	1007.0	2024-03-15	C006	Shoes	Clothing	2.0	50.0	100.000000	
7	1008.0	2024-03-18	C007	T-Shirt	Clothing	1.0	25.0	25.000000	
8	1009.0	2024-03-20	C008	Smartwatch	Electronics	1.0	200.0	200.000000	
9	1010.0	2024-04-01	C009	Book	Books	2.0	20.0	40.000000	
10	1011.0	2024-04-05	C002	Smartphone	Electronics	2.0	600.0	1200.000000	
11	1012.0	2024-04-10	C010	Tablet	Electronics	1.0	500.0	500.000000	
12	1013.0	2024-05-01	C011	Shoes	Clothing	1.0	50.0	50.000000	
13	1014.0	2024-05-05	C012	Headphones	Electronics	1.0	100.0	100.000000	
14	1015.0	2024-05-08	C013	Laptop	Electronics	1.0	800.0	800.000000	
15	1016.0	2024-05-10	C014	T-Shirt	Clothing	3.0	25.0	75.000000	
16	1017.0	2024-06-01	C015	Smartwatch	Electronics	1.0	200.0	200.000000	
17	1018.0	2024-06-05	C016	Book	Books	4.0	20.0	80.000000	
18	1019.0	2024-06-08	C017	Smartphone	Electronics	1.0	600.0	600.000000	
19	1020.0	2024-06-10	C018	Tablet	Electronics	1.0	500.0	500.000000	

In [9]:

```
#Standardize 'Date' column (all dates are in correct form)
df['Date'] = pd.to_datetime(df['Date'], errors='coerce', dayfirst=False) #to_datetime
# Format will automatically be YYYY-MM-DD when printed or saved
# 2. Clean numeric columns (if needed) so wherever there are numbers, make sure they are floats
# First, identify columns that should be numeric
```

```
numeric_cols = ['Quantity', 'Price', 'Total_Amount', 'Advertising_Spend'] #these are the columns to be converted to numeric
# Remove symbols (e.g., $, commas) and convert to float
for col in numeric_cols:
    df[col] = df[col].astype(str).str.replace(r'[^\d.]', '', regex=True) # checks for non-numeric characters and removes them
    df[col] = pd.to_numeric(df[col], errors='coerce') # make it into no. format, non-numeric values become NaN
    df[col] = df[col].round(2) #make it into 2dp

df
```

Out[9]:

	Transaction_ID	Date	Customer_ID	Product	Category	Quantity	Price	Total_Amount	F
0	1001.0	2024-01-05	C001	Laptop	Electronics	1.0	800.0	375.26	
1	1002.0	2024-01-10	C002	Smartphone	Electronics	2.0	600.0	1200.00	
2	1003.0	2024-01-12	C003	Headphones	Electronics	1.0	100.0	100.00	
3	1004.0	2024-02-05	C004	Tablet	Electronics	1.0	500.0	500.00	
4	1005.0	2024-02-08	C005	Book	Books	3.0	20.0	60.00	
5	1006.0	2024-02-10	C001	Laptop	Electronics	1.0	800.0	800.00	
6	1007.0	2024-03-15	C006	Shoes	Clothing	2.0	50.0	100.00	
7	1008.0	2024-03-18	C007	T-Shirt	Clothing	1.0	25.0	25.00	
8	1009.0	2024-03-20	C008	Smartwatch	Electronics	1.0	200.0	200.00	
9	1010.0	2024-04-01	C009	Book	Books	2.0	20.0	40.00	
10	1011.0	2024-04-05	C002	Smartphone	Electronics	2.0	600.0	1200.00	
11	1012.0	2024-04-10	C010	Tablet	Electronics	1.0	500.0	500.00	
12	1013.0	2024-05-01	C011	Shoes	Clothing	1.0	50.0	50.00	
13	1014.0	2024-05-05	C012	Headphones	Electronics	1.0	100.0	100.00	
14	1015.0	2024-05-08	C013	Laptop	Electronics	1.0	800.0	800.00	
15	1016.0	2024-05-10	C014	T-Shirt	Clothing	3.0	25.0	75.00	
16	1017.0	2024-06-01	C015	Smartwatch	Electronics	1.0	200.0	200.00	
17	1018.0	2024-06-05	C016	Book	Books	4.0	20.0	80.00	
18	1019.0	2024-06-08	C017	Smartphone	Electronics	1.0	600.0	600.00	
19	1020.0	2024-06-10	C018	Tablet	Electronics	1.0	500.0	500.00	

```
In [10]: #gets rid of extra dp in quantity
df['Quantity'] = df['Quantity'].astype(int) #astype means convert it into a certain
df
```

Out[10]:

	Transaction_ID	Date	Customer_ID	Product	Category	Quantity	Price	Total_Amount	F
0	1001.0	2024-01-05	C001	Laptop	Electronics	1	800.0	375.26	
1	1002.0	2024-01-10	C002	Smartphone	Electronics	2	600.0	1200.00	
2	1003.0	2024-01-12	C003	Headphones	Electronics	1	100.0	100.00	
3	1004.0	2024-02-05	C004	Tablet	Electronics	1	500.0	500.00	
4	1005.0	2024-02-08	C005	Book	Books	3	20.0	60.00	
5	1006.0	2024-02-10	C001	Laptop	Electronics	1	800.0	800.00	
6	1007.0	2024-03-15	C006	Shoes	Clothing	2	50.0	100.00	
7	1008.0	2024-03-18	C007	T-Shirt	Clothing	1	25.0	25.00	
8	1009.0	2024-03-20	C008	Smartwatch	Electronics	1	200.0	200.00	
9	1010.0	2024-04-01	C009	Book	Books	2	20.0	40.00	
10	1011.0	2024-04-05	C002	Smartphone	Electronics	2	600.0	1200.00	
11	1012.0	2024-04-10	C010	Tablet	Electronics	1	500.0	500.00	
12	1013.0	2024-05-01	C011	Shoes	Clothing	1	50.0	50.00	
13	1014.0	2024-05-05	C012	Headphones	Electronics	1	100.0	100.00	
14	1015.0	2024-05-08	C013	Laptop	Electronics	1	800.0	800.00	
15	1016.0	2024-05-10	C014	T-Shirt	Clothing	3	25.0	75.00	
16	1017.0	2024-06-01	C015	Smartwatch	Electronics	1	200.0	200.00	
17	1018.0	2024-06-05	C016	Book	Books	4	20.0	80.00	
18	1019.0	2024-06-08	C017	Smartphone	Electronics	1	600.0	600.00	
19	1020.0	2024-06-10	C018	Tablet	Electronics	1	500.0	500.00	

In [11]: df.to_csv(r"C:\Users\Rashid\Desktop\exported_sales_data.csv", index=False) #convert

In [12]: # Preview data
print("First 5 Rows:")
df.head()

First 5 Rows:

Out[12]:

	Transaction_ID	Date	Customer_ID	Product	Category	Quantity	Price	Total_Amount	Pa
0	1001.0	2024-01-05	C001	Laptop	Electronics	1	800.0	375.26	
1	1002.0	2024-01-10	C002	Smartphone	Electronics	2	600.0	1200.00	
2	1003.0	2024-01-12	C003	Headphones	Electronics	1	100.0	100.00	
3	1004.0	2024-02-05	C004	Tablet	Electronics	1	500.0	500.00	
4	1005.0	2024-02-08	C005	Book	Books	3	20.0	60.00	

```
In [13]: # Add calculated total= quantity*price and tot match check= makes sure price matches
df["Calculated_Total"] = df["Quantity"] * df["Price"]
df["Total_Match"] = df["Calculated_Total"].round(2) == df["Total_Amount"].round(2)

# Summary (mean,mode,median,range etc) statistics
print("Summary Statistics:\n")
print(df[["Quantity", "Price", "Total_Amount", "Advertising_Spend"]].describe(), "\n")
```

Summary Statistics:

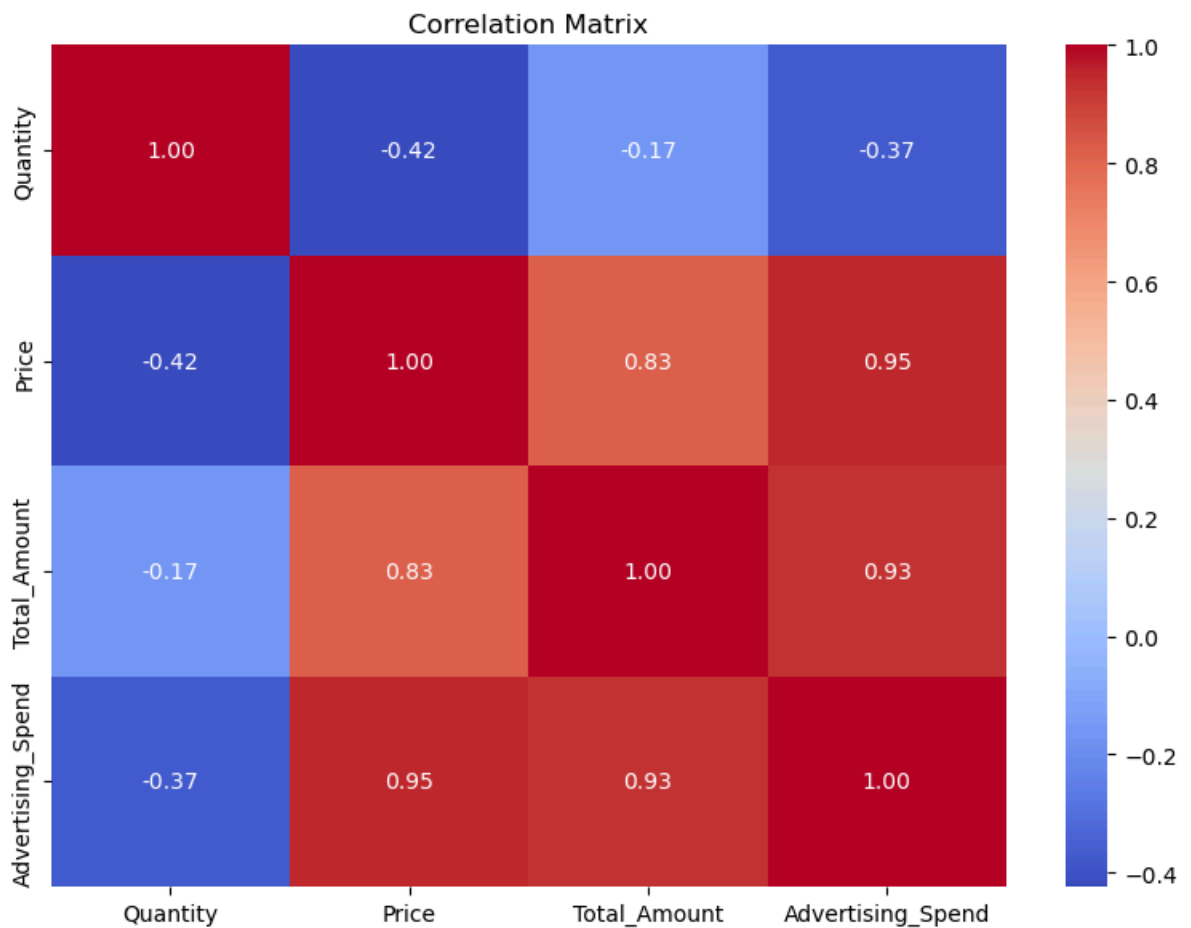
	Quantity	Price	Total_Amount	Advertising_Spend
count	20.000000	20.000000	20.000000	20.000000
mean	1.550000	325.500000	375.26300	132.750000
std	0.887041	302.484884	378.69052	106.233048
min	1.000000	20.000000	25.000000	10.000000
25%	1.000000	43.750000	78.750000	23.750000
50%	1.000000	200.000000	200.000000	120.000000
75%	2.000000	600.000000	525.000000	212.500000
max	4.000000	800.000000	1200.000000	300.000000

```
In [14]: # Correlation matrix-tells relationship betw the diff numeric col ex does quantity
correlation_matrix = df[["Quantity", "Price", "Total_Amount", "Advertising_Spend"]].corr()
print("Correlation Matrix:\n") #
print(correlation_matrix, "\n")

# Plot correlation heatmap
plt.figure(figsize=(8, 6))
sns.heatmap(correlation_matrix, annot=True, cmap="coolwarm", fmt=".2f")
plt.title("Correlation Matrix")
plt.tight_layout()
plt.savefig("correlation_matrix.png")
plt.show()
```

Correlation Matrix:

	Quantity	Price	Total_Amount	Advertising_Spend
Quantity	1.000000	-0.423792	-0.168886	-0.371559
Price	-0.423792	1.000000	0.827437	0.949682
Total_Amount	-0.168886	0.827437	1.000000	0.925503
Advertising_Spend	-0.371559	0.949682	0.925503	1.000000



```
In [15]: # EDA Summary
summary = """

1. Dataset Overview:
- 21 transactions #(means 21 rows)
- Key fields: Product, Category, Quantity, Price, Total Amount, Ad Spend, Region, D

2. Key Trends and Patterns:
- Most transactions involve 1 unit (Median Quantity = 1)
- Average Price: £328, ranging from £20 to £800
- Total Amount: £25 to £1200
- Ad Spend: £10 to £375 (Average ≈ £144)

3. Correlations:
- Price vs. Total_Amount: Strong (0.93)
- Quantity vs. Total_Amount: Moderate (0.44)
- Advertising_Spend vs. Total_Amount: Weak (0.11)

""" #""""needed which means multi line so text is written on multiple lines

print(summary)
```

1. Dataset Overview:

- 21 transactions #(means 21 rows)
- Key fields: Product, Category, Quantity, Price, Total Amount, Ad Spend, Region, Date

2. Key Trends and Patterns:

- Most transactions involve 1 unit (Median Quantity = 1)
- Average Price: £328, ranging from £20 to £800
- Total Amount: £25 to £1200
- Ad Spend: £10 to £375 (Average ≈ £144)

3. Correlations:

- Price vs. Total_Amount: Strong (0.93)
- Quantity vs. Total_Amount: Moderate (0.44)
- Advertising_Spend vs. Total_Amount: Weak (0.11)

```
In [16]: # Add a Month column for grouping- chose month for the line graph
df['Month'] = df['Date'].dt.to_period('M') #to_period means converting to a time pe

# Set style
sns.set(style="whitegrid") #sns=heatmap, style to be a white grid

# Line Graph: total Sales over the month
monthly_sales = df.groupby('Month')['Total_Amount'].sum().reset_index() #we are gro
monthly_sales['Month'] = monthly_sales['Month'].astype(str) #we want the month writ

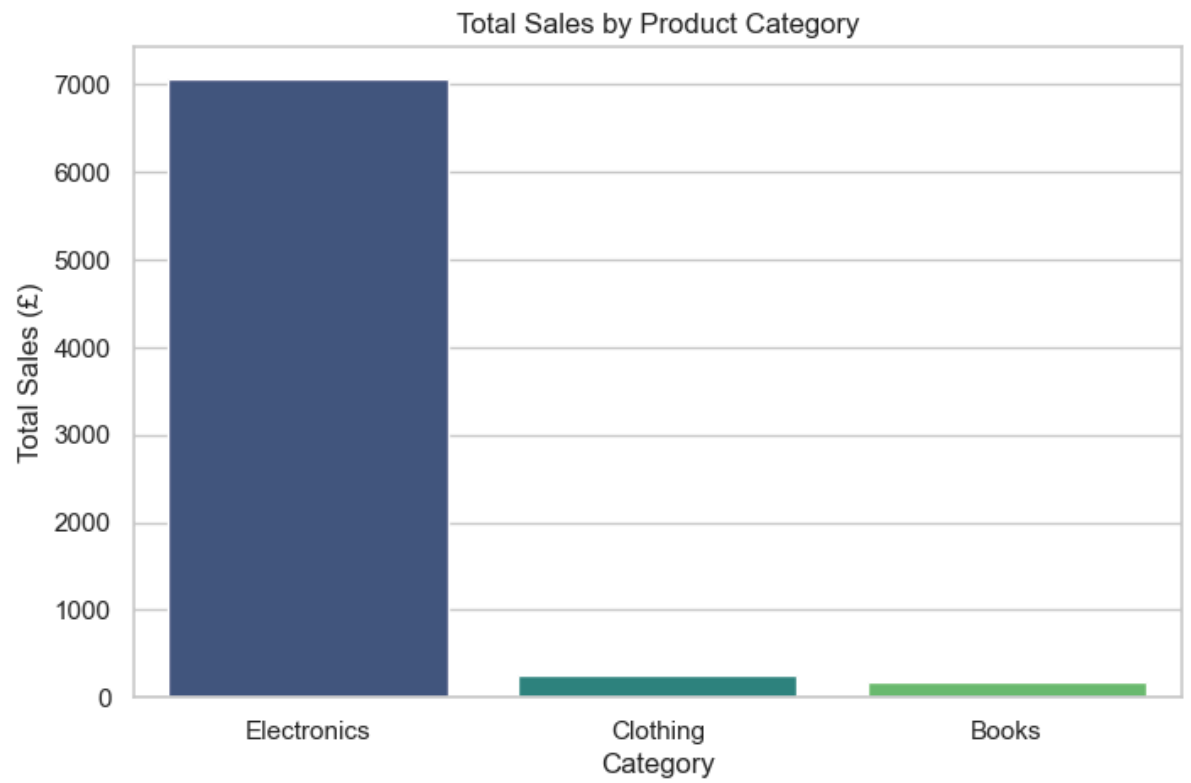
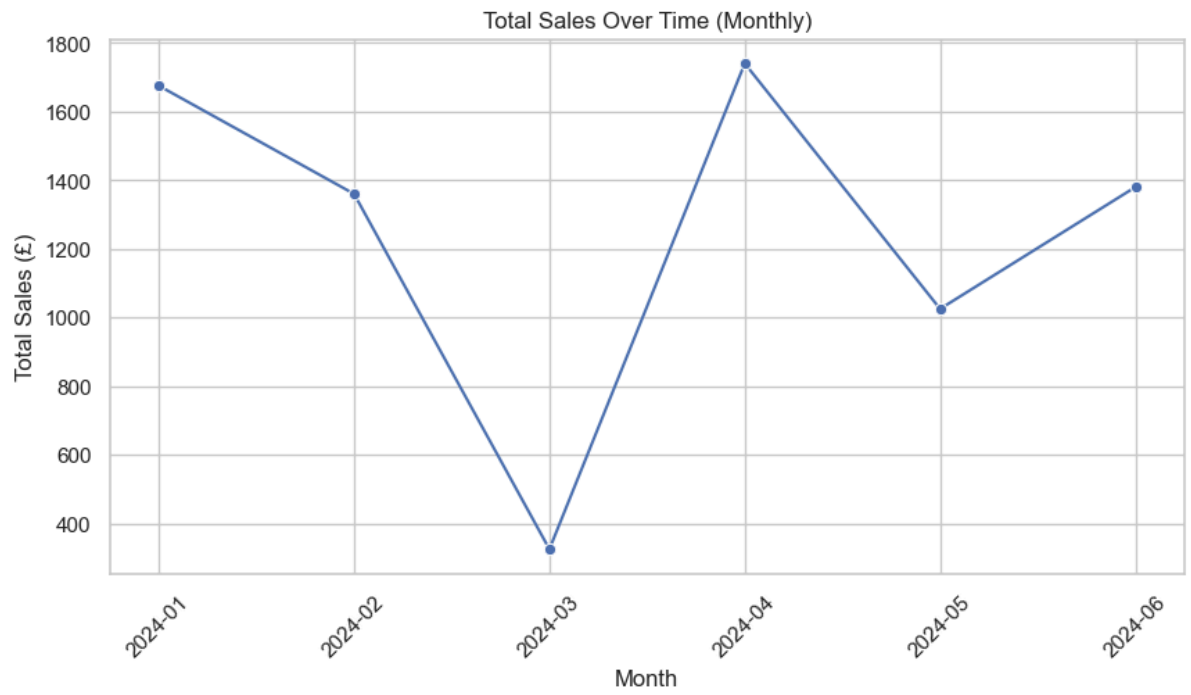
plt.figure(figsize=(10, 5)) #plt= plot,fig size i can make up
sns.lineplot(data=monthly_sales, x='Month', y='Total_Amount', marker='o') #marker".
plt.title("Total Sales Over Time (Monthly)")
plt.xlabel("Month")
plt.ylabel("Total Sales (£)")
plt.xticks(rotation=45) #rotates x axis text (months) by how many degrees i want
plt.show()

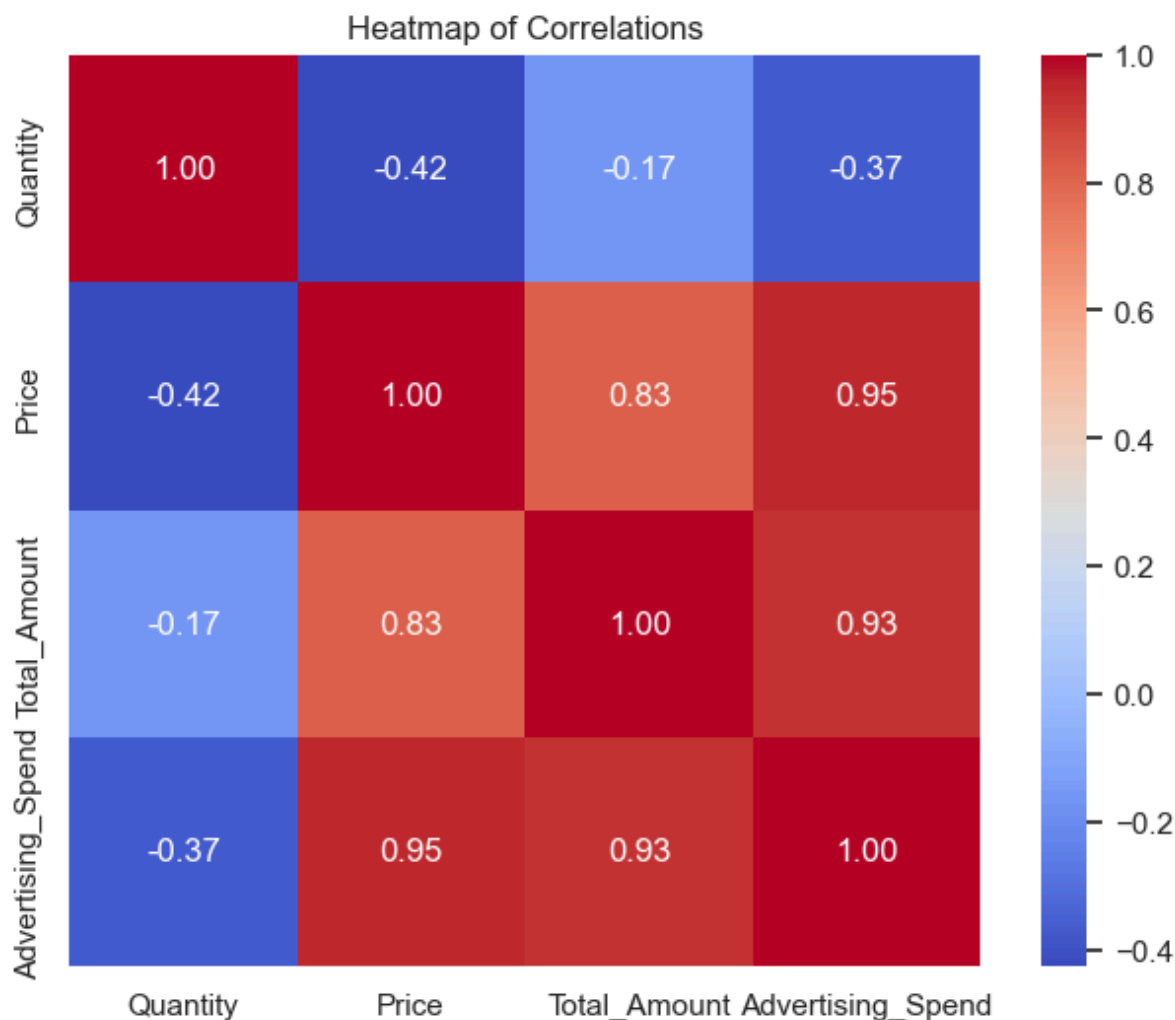
# 2. Bar Chart: Sales by Product Category- Bar chart comparing the sales of differ
category_sales = df.groupby('Category')['Total_Amount'].sum().reset_index().sort_va

plt.figure(figsize=(8, 5)) #i can choose
sns.barplot(data=category_sales, x='Category', y='Total_Amount', palette='viridis')
plt.title("Total Sales by Product Category")
plt.xlabel("Category")
plt.ylabel("Total Sales (£)")
plt.show()

# 3. Heatmap: Correlation Matrix-Correlation matrix heatmap to show relationships b
correlation_matrix = df[["Quantity", "Price", "Total_Amount", "Advertising_Spend"]].

plt.figure(figsize=(8, 6))
sns.heatmap(correlation_matrix, annot=True, cmap="coolwarm", fmt=".2f", square=True)
plt.title("Heatmap of Correlations")
plt.show()
```





```
In [17]: #4) Final Data Insights Report
#1) Summarise Key Findings: Begin your report by summarising the most important insights
#- What are the best-selling months or product categories?- Are there any correlations?
# - What customer behaviors or purchasing trends were identified? NEED TO BE IDENTIFIED
#2) Provide Actionable Recommendations: Based on the analysis, offer recommendations
#that can help the business improve sales. This could include:
#- Focusing marketing efforts on peak sales periods.
#- Increasing stock for best-selling products.
#- Adjusting sales strategies for slow-performing months.
#3) Ethical Considerations - Mention any ethical issues, such as the importance of
#anonymising customer data and ensuring compliance with privacy laws (e.g., GDPR).

summary = """
The best selling months is April with the highest total sales of around 1750 pounds
The best selling product category was the electronics with above 7000 pounds worth
The purchasing trends
Based on the analysis, shopease can increase marketing or find alternative sales strategies
and asking them to review the product, if they give a low rating score from 0-5 star
customer is not satisfied with product. For example the majority of the total sales
clothing and books have a low amount for total sales amount in pounds so for customer
follow them up and ask them to review their product. Then we can find out why the customer
much as the electronics.
For the slow performing months as well as using customer reviews to find out why the
so for the months where there was a decline in sales amount in pounds from January
and from February to March and from April to May. We could also use discounts like
which would increase total sales amount in pounds for shopease.

so from January to February and from February to March and from April to May and in
the majority of total sales amount in pounds comes from electronics, so to keep this
to keep the customers purchasing from shopease. On peak sales periods which are March
Since there is an increase in total sales in pounds from the line graph.
```

Customer Data Privacy:

All customer data used in the analysis has been anonymised to protect individual id in compliance with data protection regulations such as GDPR.

Compliance and Transparency:

The business must ensure ongoing adherence to privacy laws and ethical standards by practices and securing informed consent where required.

Responsible Use of Data:

Insights derived from customer behavior should be used to enhance customer experier information or engaging in manipulative marketing tactics."""

In []:

In []:

In []:

In []:

In []:

In []:

In []:

In []:

In []: